



INTRODUCTION TO VUEJS

VUE.JS IS AN APPROACHABLE, PERFORMANT, AND VERSATILE JAVASCRIPT FRAMEWORK USED FOR BUILDING INTUITIVE USER INTERFACES AND MODERN SINGLE-PAGE APPLICATIONS.



Welcome 🙌

Let's get to know each other

What's your name?

Where do you work? Do you have any experience in it?

What are your expectations of this course?

INTRODUCTION



CREATING YOUR OWN VUEJS PROJECT

To Start you need to scaffold a professional VueJS development environment on your computer. We will use **Vite**, the official build tool recommended by the Vue team. Vite creates a modern project with Hot Module Replacement, syntax highlighting, and an instant dev server. You generate a fresh one using a terminal command

Steps to create VueJS projects:

>> Step 1: Create a new Vue project (replace 'my-vue-app' with your project name)

```
npm create vue@latest my-vue-app
```

>> Step 2: Navigate into the project folder

```
cd my-vue-app
```

>> Step 3: Install dependencies

```
npm install
```

>> Step 4: Start the development server

```
npm run dev
```

PROJECT TASK



CREATING YOUR OWN VUEJS PROJECT

1. **Action:** Open your terminal (Command Prompt, PowerShell, or VS Code terminal).
2. **Instructions:**
 - >> Download NodeJs on your machine
 - >> Run : `npm create vue@latest my-todo-app`
(choose "Yes" for Vue Router and Pinia if asked, though we won't use them yet).
 - >> Run : `cd my-todo-app`
 - >> Run : `npm install.`
 - >> Run : `npm run dev.`
 - >> Click the localhost link (usually `http://localhost:5173`) to see your brand new Vue welcome page.
3. **Verification:** You should see the official Vue starter template with the Vite logo. You have successfully created a VueJS project from nothing

INTRODUCTION



WHAT ARE UI FRAMEWORKS?

Before diving into code, we must understand the "why."

A **UI Framework** (or Library) is a **pre-written collection of JavaScript code** that helps you **build complex, interactive user interfaces** without manually manipulating every single HTML element. Instead of writing `document.getElementById` and `innerHTML` for every change, frameworks handle the heavy lifting of updating the screen when your data changes

➤ Example:

// Without a Framework (Vanilla JS)

```
const title = document.getElementById('title');  
title.innerHTML = 'New Text';
```

// With a Framework (VueJS)

```
// When 'message' changes, the <p> updates automatically.  
// data.message = 'New Text'; // UI refreshes automatically
```

PROJECT TASK

UI FRAMEWORKS

1. File to Open: **package.json** (in your newly created project)
2. Action: Look for **"vue"** inside the "dependencies" object.
 >> This confirms your project is a VueJS application.
1. Modification: Delete the default **src/components/HelloWorld.vue** file and **src/assets/logo.svg** to clean your project for our Todo app.
2. Result: You now have an empty, clean VueJS project ready for building



INTRODUCTION



MEET THE BIG THREE – REACT, ANGULAR, VUE

React	
Library	META / Facebook
Component-based library. Unidirectional data flow. JSX syntax. Large ecosystem. Most market share. Best For: High-performance, scalable apps (Instagram, Airbnb)	

Angular	
Framework	Google
Full opinionated framework. TypeScript-first. Two-way binding. Steep learning curve. Enterprise focus. Best For: Large enterprise apps (Google Cloud, Upwork)	

Vue	
Progressive	Community-driven
Progressive framework. Gentle learning curve. Two-way binding. Flexible integration. Our focus Best For: Interactive interfaces, teams learning gradually →	

➤ Example:

```
// React (JSX)
function Welcome() { return <h1>Hello {name}</h1>; }
```

```
// Angular (TypeScript with Decorators)
@Component({ template: '<h1>Hello {{name}}</h1>' })
```

```
// Vue (Single File Component – our project)
<template><h1>Hello {{ name }}</h1></template>
```

PROJECT TASK



MEET THE BIG THREE — REACT, ANGULAR, VUE

No coding yet. Simply explore the project structure. Open these files in your editor and observe what they contain:

>> `index.html` — Find `<div id="app">`

>> `src/main.js` — Find `createApp(App).mount('#app')`

>> `src/App.vue` — Notice the three sections: `<template>`, `<script>`, `<style>`

>> `package.json` — Find `"vue"` in dependencies

INTRODUCTION



GUIDED TOUR – VUE PROJECT STRUCTURE

Every Vite-generated Vue project follows a standard structure.

The entry point is **main.js**, which creates the Vue app and mounts it to **<div id="app">** in **index.html**.

All your components live in the **src/components/ folder**. The main layout is **App.vue**.

Understanding this flow is essential: **index.html** → **main.js** → **App.vue** → **Your Components**

Every Vite-generated Vue project follows a consistent structure that separates concerns:

my-vue-app/	
— index.html	# Entry point - contains <div id="app">
— src/	
— main.js	# Creates Vue app and mounts to #app
— App.vue	# Root component (all others nest inside)
— components/	# Reusable UI pieces
— assets/	# CSS, images, fonts
— package.json	# Dependencies and scripts
— vite.config.js	# Build configuration

➤ **Example:** Have a look on the **main.js** file generated by Vite

PROJECT TASK

VUEJS PROJECT STRUCTURE

1. Delete the default `src/assets/base.css`
2. Instructions:
 >> in `src/assets/` paste the `styles.css` file provided as materials
3. Modification: Update the `src/main.js` to import the new CSS



INTRODUCTION



THE PROGRESSIVE NATURE OF VUE

Vue is called "progressive" because you can adopt it incrementally. You are not forced to use the full toolchain:

>> CDN script tag :

When to use?

Adding interactivity to a static HTML page (like jQuery replacement)

>> Single File Components (our project):

When to use?

Building a full Single Page Application

>> Vue Router + Pinia:

When to use? Complex applications with routing and global state

This means you can start small and scale up as requirements grow—unlike Angular, which requires full commitment upfront

INTRODUCTION



THE PROGRESSIVE NATURE OF VUE - EXAMPLE

<!-- Level 1: Vue via CDN - No build tools needed -->

<!DOCTYPE html>

<html>

<body>

<div id="app">

<input v-model="message" placeholder="Type something">

<p>You typed: {{ message }}</p>

</div>

<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>

<script>

Vue.createApp({

data()

{

return { message: "" }

}

}).mount('#app')

</script>

</body>

</html>

PROJECT TASK

THE PROGRESSIVE NATURE OF VUE

1. **Action: Create a folder called demo:**
 - >> Open it in VSCode
 - >> Create a file called `vue-cdn-demo.html`
2. **Instructions: Paste the code above and open it in your browser.**
3. **Right click on the file and open it with Live Server**
4. **Result: You see a working Vue app with two-way binding, created without npm, Vite, or any build step. This is the "progressive" entry point**



INTRODUCTION



YOUR FIRST VUE COMPONENT — FRAMEWORKSOVERVIEW

Now you will create your first Vue component.

This component will display information about React, Angular, and Vue **using v-for** to loop through an **array of framework objects**.

This demonstrates **declarative rendering** — you define the data, and Vue generates the HTML

➤ **Note:**

- **v-for** — loops through an array to generate multiple elements
- **:key** — unique identifier for each item (helps Vue with updates)
- **{{ }}** — interpolation (displays data in template)
- **:class** — dynamic class binding (adds 'highlight' class to Vue card)

INTRODUCTION



YOUR FIRST VUE COMPONENT – EXAMPLE

```
<template>
  <div v-for="fw in frameworks" :key="fw.name">
    <h3>{{ fw.name }}</h3>
    <p>{{ fw.description }}</p>
  </div>
</template>

<script setup>
  const frameworks = [
    { name: 'React', description: 'UI library' },
    { name: 'Angular', description: 'Full framework' },
    { name: 'Vue', description: 'Progressive' }
  ]
</script>
```

PROJECT TASK

YOUR FIRST VUE COMPONENT – FRAMEWORKSOVERVIEW

Create `src/components/FrameworksOverview.vue` that displays framework cards using `v-for`

Then import and use it in `App.vue`



INTRODUCTION



DECLARATIVE RENDERING – VUE'S CORE MAGIC

Now that we understand what frameworks do, let's see Vue's core mechanism: **declarative rendering**. You define **reactive data** in JavaScript, and Vue **automatically reflects changes** in the DOM using `{{ }}` interpolation (mustache syntax) and **directives** like **v-bind** (or its shorthand `:`). This eliminates the need for manual DOM queries.

➤ **Such as:**

- >> `ref()` – Creates reactive data
- >> `{{ }}` – Interpolation (displays reactive data in the template)
- >> `@click` – Event listener

➤ **Example:**

```
<template>
  <div>
    <h1>{{ title }}</h1>
    <p>{{ message }}</p>
    <button @click="count++">Clicked {{ count }} times</button>
  </div>
</template>
<script setup>
  import { ref } from 'vue'
  const title = ref('Welcome to VueJS')
  const message = ref('This updates automatically!')
  const count = ref(0)
</script>
```

PROJECT TASK

DECLARATIVE RENDERING – VUE'S CORE MAGIC

Create a simple task that uses interactive component that demonstrates declarative rendering.

Create reactive variables for a **title**, a **subtitle**, and a **counter**. Display them and make the counter increment when a button is clicked

(The template and Style will be provided to you as Materials)



INTRODUCTION



REACT VS ANGULAR

➤ The main difference is data flow:

➤➤ **React:** One-way binding. Data flows parent → child via props. Child updates parent via callbacks. Predictable but more code.

➤➤ **Angular:** Two-way binding with [(ngModel)]. UI and data sync automatically. Convenient but can be harder to debug.

Vue offers both — you choose. We'll learn v-model (two-way)

➤ Example:

// Vue: Both available

```
<input :value="text" @input="text = $event.target.value" /> <!-- One-way -->
```

```
<input v-model="text" /> <!-- Two-way -->
```

➤ Q & A:

When would you choose one-way binding?

➤➤ When you want explicit, traceable data flow (large apps, many developers)

When would you choose two-way binding?

➤➤ For simple forms where convenience matters more than predictability

INTRODUCTION



REACT VS ANGULAR - EXAMPLE

<!-- Vue's sweet spot: familiar HTML + reactive power -->

```
<template>
  <div>
    <input v-model="search" placeholder="Search..." />
    <p v-if="search">Searching for: {{ search }}</p>
    <li v-for="item in filteredItems" :key="item.id">
      {{ item.name }}
    </li>
  </div>
</template>
```

INTRODUCTION



WHY VUEJS?

KEY FEATURE	 React	 Angular	 Vue.js
Learning Curve	 Moderate	 Steep	 Gentle
Two-way Binding	 No	 Yes	 Yes <code>v-model</code>
Single File Components	 No <code>JSX Format</code>	 Yes	 Yes
Official Router & State	 No <i>User choice (Redux, Router)</i>	 Yes	 Yes <i>Optional (Vue Router / Pinia)</i>

PROJECT TASK

WHY VUEJS?

No coding. Look ahead at the final project features:

➤ Detect where could we have used the following VueJS features:

- >> Two-way binding
- >> List rendering
- >> Conditional output



INTRODUCTION



THE VUE APPLICATION INSTANCE - MAIN.JS

- Every Vue app starts with **createApp()**, which creates the app instance.
- **mount()** tells Vue where to inject the rendered HTML into the DOM.
- The **main.js** file is the **entry point** — it's executed when your page loads

PROJECT TASK

THE VUE APPLICATION INSTANCE - MAIN.JS

Update `src/main.js` to match the final project version. Add comments explaining each line. Ensure it imports the correct CSS file



INTRODUCTION



UNDERSTANDING REF AND REACTIVE DATA

`ref()` is the foundation of Vue's reactivity system.

It creates a reactive reference (wrapper) around any value.

When you change the `.value` property, Vue detects the change and re-renders any template that uses that ref.

>> **Why `.value`?** JavaScript can't make primitive values (numbers, strings) reactive. Wrapping in an object with `.value` allows Vue to track changes

➤ **Note:**

>> In `<script>` — MUST use `.value` (e.g., `count.value++`)

>> In `<template>` — NO `.value` (e.g., `{{ count }}`)

➤ **Example:**

```
import { ref } from 'vue'
const count = ref(0) // Ref<number>
const user = ref({ name: 'John' }) // Ref<object>
```

// In script using functions — MUST use `.value`

```
count.value = 10
```

```
user.value.name = 'Jane'
```

// In template — NO `.value`

```
// <p>{{ count }}</p>
```

```
// <p>{{ user.name }}</p>
```

PROJECT TASK

UNDERSTANDING REF AND REACTIVE DATA

In **App.vue**

- >> create reactive state for the todo app: todos array (empty), filterText string (empty).
- >> Add a temporary debug display to see the reactivity



INTRODUCTION



THE V-ON DIRECTIVE - CLICK, INPUT, KEYUP

v-on listens to DOM events. The shorthand **@** is recommended. You can write inline expressions or call methods.

➤ Common event modifiers:

- >> **.prevent** — calls `event.preventDefault()`
- >> **.stop** — calls `event.stopPropagation()`
- >> **.enter** — triggers on Enter key
- >> **.once** — triggers only once

➤ Example:

```
<template>  
  <button @click="count++">+1</button>  
  <button @click="decrement">-1</button>  
  <input @keyup.enter="submit" />  
  <form @submit.prevent="handleSubmit"> <!-- Prevents page reload -->  
</template>
```

PROJECT TASK



THE V-ON DIRECTIVE - CLICK, INPUT, KEYUP

Create `src/components/ToDoInput.vue` exactly as in the final project. The component must:

- >> Use `ref()` for `newTitle`
- >> Listen to `@input` to update `newTitle`
- >> Listen to `@keyup.enter` to add todo
- >> Listen to `@click` on button to add todo
- >> Emit `add-todo` event with the title

INTRODUCTION



EMITTING EVENTS TO PARENTS : DEFINEEMITS

Components communicate using props down, events up:

- >> Child emits an event to tell parent something happened
- >> Parent listens with @event-name

defineEmits() declares which events a component can emit. This provides type safety and documentation

➤ Example:

```
<!-- Child component -->
<script setup>
  const emit = defineEmits(['add-todo', 'close'])
  emit('add-todo', { title: 'New todo' })
</script>

<!-- Parent component -->
<template>
  <TodoInput @add-todo="handleAdd" />
</template>

<script setup>
  const handleAdd = (todoData) => {
    console.log('Received from child:', todoData)
  }
</script>
```

PROJECT TASK

EMITTING EVENTS TO PARENTS : DEFINEEMITS

>> In **App.vue**, implement **handleAdd** that adds the new todo to the todos array.

>> Also add the **TodoInput** component to the template



INTRODUCTION



TWO-WAY BINDING WITH V-MODEL : TODOFILTER

>> **v-model** is syntactic sugar for **:value** + **@input**.

Instead of writing: `<input :value="text" @input="text = $event.target.value" />`

You write: `<input v-model="text" />`

v-model automatically keeps the data and the input in sync

➤ Example:

```
<template>
  <!-- Text input -->
  <input v-model="username" />

  <!-- Checkbox (boolean) -->
  <input type="checkbox" v-model="agreed" />

  <!-- Checkbox group (array) -->
  <input type="checkbox" value="vue" v-model="frameworks" /> Vue
</template>
```


PROJECT TASK

TWO-WAY BINDING WITH V-MODEL : TODOFILTER

Create `src/components/ToDoFilter.vue` exactly as in the final project.

The component must:

- >> Use `v-model` on an input bound to `filterText`
- >> Display a hint showing the current filter value
- >> Use `watch` to emit changes to the parent



INTRODUCTION



OUTPUTTING LISTS WITH V-FOR : TODOLIST

>> **v-for** renders a list of items.

Always provide a **key** — it helps Vue track each item's identity for efficient updates (reordering, adding, removing).

>> Syntax:

v-for="item in items"

or

v-for="(item, index) in items"

➤ Example:

```
<template>
```

```
  <!-- Basic list -->
```

```
  <li v-for="todo in todos" :key="todo.id">
```

```
    {{ todo.title }}
```

```
  </li>
```

```
  <!-- With index -->
```

```
  <li v-for="(todo, index) in todos" :key="todo.id">
```

```
    {{ index + 1 }}. {{ todo.title }}
```

```
  </li>
```

```
</template>
```

PROJECT TASK

TWO-WAY BINDING WITH V-MODEL : TODOFILTER

Create `src/components/ToDoList.vue` that:

- >> Receives `todos` as a prop
- >> Uses `v-for` to render each todo
- >> Displays a checkbox and the todo title
- >> Emits `toggle-todo` and `delete-todo` events



INTRODUCTION



OUTPUTTING CONTENT CONDITIONALLY USING VUEJS

Vue gives you three directives to conditionally show or hide elements in the template. Use them to control what is rendered based on your reactive data.

➤ **The three conditional directives:**

- **v-if** — Adds or removes the element from the DOM based on a condition
- **v-else-if** — Chains an additional condition after v-if
- **v-else** — Fallback rendered when all v-if / v-else-if conditions are false

➤ **Example:**

```
<div v-if="isLoading">Loading...</div>  
<div v-else-if="error">Error: {{ error }}</div>  
<div v-else>Content loaded successfully!</div>
```

PROJECT TASK



OUTPUTTING CONTENT CONDITIONALLY USING VUEJS

In App.vue, the loading and error states are already wired using conditional directives. Identify and understand each one:

- >> **v-if="isLoading"** — show a spinner while data is being fetched
- >> **v-else-if="error"** — show the error message if the fetch failed
- >> **v-else** — show the full todo UI when data is ready

Verification: Temporarily set isLoading to true in useTodos.js and confirm the spinner appears instead of the list.

INTRODUCTION



UPDATING AND DELETING DATA USING VUEJS

Once data is rendered, users need to modify it. In Vue, updating and deleting data means changing the reactive array that drives the template. Vue automatically re-renders the list when the array changes.

➤ **Key patterns:**

➤ **Update (toggle):** Find the item by id and mutate its property directly

➤ **Delete:** Replace the array with a filtered copy using `.filter()`

➤ **Example:**

`// Toggle completion`

`const todo = todos.value.find(t => t.id === id)`

`todo.completed = !todo.completed`

`// Delete an item`

`todos.value = todos.value.filter(t => t.id !== id)`

PROJECT TASK



UPDATING AND DELETING DATA USING VUEJS

Complete the toggle and delete logic in App.vue to allow users to check off and remove todos from the list.

- >> **handleToggle(id)**: Find the matching todo in the todos array and flip its completed property
- >> **handleDelete(id)**: Filter out the todo with the matching id from todos.value

Make sure TodoList.vue emits **toggle-todo** and **delete-todo** events, and that App.vue listens to both with **@toggle-todo** and **@delete-todo**.

Verification: Clicking the checkbox should cross out the todo; clicking Delete should remove it from the list instantly.

INTRODUCTION



SENDING POST HTTP REQUESTS USING VUEJS

To send data to a backend, use the browser's built-in **fetch()** API with method: 'POST'. The body must be serialized to JSON and the **Content-Type** header must be set.

➤ **Key concepts:**

- **async/await** — waits for the network response before continuing
- **JSON.stringify()** — converts a JS object into a JSON string for the request body
- **response.json()** — parses the server response back into a JS object

➤ **Example:**

```
const res = await fetch('/api/todos', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ title, completed: false })  
})  
const created = await res.json()
```

PROJECT TASK



SENDING POST HTTP REQUESTS USING VUEJS

Create **src/composables/useTodoPost.js** and wire it into **App.vue** so that adding a todo sends a real POST request.

>> **postTodo(title)**: POST to <https://jsonplaceholder.typicode.com/todos> with { title, completed: false, userId: 1 }

>> **handleAdd(title) in App.vue**: call **postTodo**, then prepend the returned object to **todos.value** using **.unshift()**

Note: JSONPlaceholder is a mock API — it returns a fake created object but does not actually persist data. This is perfect for practising real HTTP requests without a real backend.

Verification: Open the browser DevTools Network tab, add a todo, and confirm a POST request appears with a 201 status code.

INTRODUCTION



LOADING DATA AND MANAGING LOADING STATES IN VUE JS

We will fetch data from an API when a component mounts. Vue provides **onMounted()** to run code after the component is inserted into the DOM. While waiting, you must track **isLoading** and **error** states so the UI gives feedback to the user.

➤ The three states to track:

- **isLoading** — true while the request is in flight; show a spinner
- **error** — holds the error message if the fetch fails; show a warning
- **data** — the fetched result; render it when ready

➤ Example:

```
import { ref, onMounted } from 'vue'
const todos = ref([]); const isLoading = ref(true)
onMounted(async () => {
  try { todos.value = await (await fetch('/api/todos')).json() }
  finally { isLoading.value = false } })
```


PROJECT TASK



LOADING DATA AND MANAGING LOADING STATES IN VUEJS

Create **src/composables/useTodos.js** that fetches todos from JSONPlaceholder on mount and exposes reactive loading state.

- » **todos, isLoading, error** — three reactive refs exposed from the composable
- » **fetchTodos()** — called inside `onMounted()`; sets `isLoading` to `true`, fetches 5 todos with `?_limit=5`, then sets `isLoading` to `false` in `finally{}`
- » **error handling** — catch any network failure and store the message in `error.value`

Import and use **useTodos()** in `App.vue` to replace any hardcoded data.

Verification: The app shows a loading spinner briefly, then displays the fetched todos. Disconnecting from the internet should show the error message.

INTRODUCTION



LOADING VUE DIFFERENTLY

Vue is “progressive” — you choose how much of it to use. There are three main ways to load Vue, each suited to a different use case:

➤ **Method 1 — CDN Script Tag:**

Drop a single **<script>** tag into any HTML page. No build tools needed. Best for adding reactivity to an existing static page.

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
```

➤ **Method 2 — Vite + Single File Components (our project):**

The professional workflow. Each component lives in its own **.vue** file with **<template>**, **<script setup>**, and **<style>**. Hot Module Replacement and tree-shaking included.

➤ **Method 3 — Vue Router + Pinia (advanced):**

Adds client-side routing and global state management on top of Vite. Used for large applications with multiple pages and shared data stores.

INTRODUCTION



SENDING PATCH AND DELETE HTTP REQUESTS IN VUEJS

Beyond creating data (POST), REST APIs use **PATCH** to update specific fields and **DELETE** to remove a resource by ID. Both use the same **fetch()** API, just with a different method string and URL.

➤ **Key difference – PATCH vs PUT:**

PATCH sends only the fields you want to change (partial update)

PUT replaces the entire resource (full update)

➤ **Example – PATCH:**

```
await fetch(`/todos/${id}`, {  
  method: 'PATCH', headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ completed: true })  
})
```

➤ **Example – DELETE:**

```
await fetch(`/todos/${id}`, { method: 'DELETE' })
```

PROJECT TASK



SENDING PATCH AND DELETE HTTP REQUESTS IN VUEJS

Create **src/composables/useTodoCRUD.js** and connect it to the toggle and delete handlers in App.vue.

- >> **patchTodo(id, updates)**: PATCH to .../todos/{id} with the updates object; throw if response is not ok
- >> **deleteTodo(id)**: DELETE to .../todos/{id}; throw if response is not ok

Update **handleToggle(id)** in App.vue to call **patchTodo** before flipping completed locally.

Update **handleDelete(id)** to call **deleteTodo** before filtering the array.

Verification: Open DevTools Network tab. Checking a todo fires a PATCH request; deleting fires a DELETE request — both should return 200 OK.